

Traffic Rule Violation Detection System

AID 728 — Computer Vision Course Project

Team 24

Shivansh Shah BT2024243

Advaya Bhardwaj BT2024231

Aman Lahoti BT2024123

IIIT Bangalore

May 17, 2026

1 Introduction

Traffic violations involving two-wheelers — particularly riding without helmets and triple-riding — are a leading cause of road fatalities in India. Manual enforcement is labour-intensive and inconsistent. This project presents an automated computer vision pipeline that, given a single RGB street image, (i) detects all two-wheelers, (ii) counts riders and identifies helmet violations, and (iii) extracts the license plate text of violating vehicles.

The system operates under strict constraints: a total model-size budget of 250 MB, no large vision-language models (>1B parameters), fully offline execution, and reasonable per-image inference time. Our approach uses a two-model YOLO architecture augmented with geometric reasoning and multi-variant OCR to achieve robust detection across challenging real-world conditions.

2 System Architecture

The pipeline consists of seven sequential stages, illustrated in Figure 1.

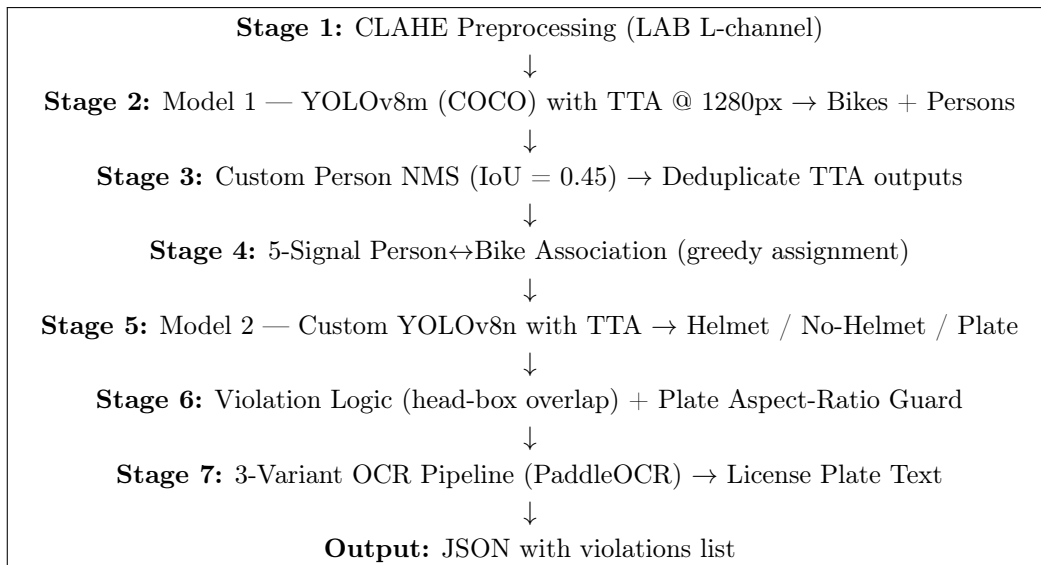


Figure 1: End-to-end pipeline overview. Each stage feeds into the next; no feedback loops.

2.1 Stage 1: CLAHE Preprocessing

Before any inference, the input image undergoes Contrast Limited Adaptive Histogram Equalisation (CLAHE) on the L-channel of LAB colour space. This normalises exposure across the image, significantly improving detection in dark, overexposed, or unevenly lit street scenes.

Listing 1: CLAHE on LAB L-channel

```
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(lab)
l = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(l)
result = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)
```

2.2 Stage 2: Two-Wheeler and Person Detection (Model 1)

A COCO-pretrained **YOLOv8m** (medium) model detects motorcycles and persons. We run inference at `imgsz=1280` ($2\times$ default resolution) with test-time augmentation (`augment=True`), which performs multi-scale and horizontal-flip inference with result merging. This catches small and distant vehicles that would be missed at 640px.

Both COCO class IDs `motorcycle` (3) and `bicycle` (1) are accepted, because COCO models sometimes label scooters and mopeds as bicycles. Confidence thresholds are set at 0.35 for bikes and 0.40 for persons.

The system includes a graceful fallback to YOLOv8s if the medium model is unavailable.

2.3 Stage 3: Person NMS

Test-time augmentation at high resolution can produce duplicate overlapping bounding boxes for the same person, which would inflate rider counts. A custom greedy NMS pass (IoU threshold = 0.45) is applied specifically to person detections to eliminate these duplicates, processing largest-area boxes first.

2.4 Stage 4: 5-Signal Person↔Bike Association

Simple IoU-based matching is insufficient for rider association because persons and bikes have different aspect ratios and only partially overlap. We introduce a **5-signal additive scoring** system:

Table 1: Association scoring signals

Signal	Weight	Description
IoU	variable	Baseline spatial overlap between person and bike boxes
Feet proximity	+0.40	Person’s bottom centre point is near the bike box (margin 0.3)
Horizontal alignment	+0.15	Person centre- x within $0.70\times$ bike width of bike centre- x
Size similarity	+0.10	Person/bike area ratio between 0.40 and 2.00
Vertical containment	+0.10	Person’s midpoint vertically intersects the bike box

Before scoring, a **3-gate fast rejection** filters out implausible pairings:

1. **Area ratio gate:** Person area $< 15\%$ of bike area $\rightarrow$ reject (background pedestrian).
2. **Vertical feet gate:** Person’s feet not within bike’s vertical range $\pm 30\%$ $\rightarrow$ reject.

3. **Torso $\cap$ seat gate:** Lower 60% of person doesn't intersect upper 70% of bike $\rightarrow$ reject.

Assignment is greedy (highest-score-first), with each person assigned to at most one bike and a maximum of 4 riders per bike. The minimum score threshold is 0.15.

2.5 Stage 5: Helmet and Plate Detection (Model 2)

A custom-trained **YOLOv8n** (nano) model detects three classes: **helmet** (0), **no_helmet** (1), and **license_plate** (2). This model is also run with TTA enabled. Plate detections are further filtered by an aspect-ratio guard ($1.5 \leq \text{width/height} \leq 8.0$) to reject false positives.

2.6 Stage 6: Violation Logic

For each rider associated with a bike, the **top 40%** of their bounding box is extracted as the “head region.” A rider is counted as a helmet violation if:

- A **no_helmet** detection overlaps their head region, **OR**
- **No helmet** detection overlaps their head region.

A bike is flagged as a violation if **num_riders** > 2 **or** **helmet_violations** > 0. This conservative approach (assume violation when uncertain) is appropriate for a safety-critical application.

2.7 Stage 7: 3-Variant OCR Pipeline

For each violating bike, the nearest license plate crop (from the original unprocessed image for better colour fidelity) is passed through PaddleOCR with three preprocessing variants, tried in sequence until a valid result (≥ 4 alphanumeric characters) is obtained:

Table 2: OCR preprocessing variants (fallback chain)

Variant	Preprocessing	Best For
v1	Upscale only	Clear, well-lit plates
v2	Upscale + CLAHE (clipLimit=3.0)	Low-contrast or dirty plates
v3	Upscale (160px) + Unsharp mask + Adaptive threshold	Blurry, tiny, or low-resolution plates

A **row-grouping sort** is applied to OCR output lines before joining: lines whose vertical ranges overlap by $\geq 40\%$ of the shorter line's height are grouped into the same row, then rows are sorted top-to-bottom and lines within each row left-to-right. This prevents multi-row plate text from being scrambled by naive (c_y, c_x) sorting.

3 Models

Table 3: Models used and size budget

Model	Architecture	Purpose	Size	Source
Model 1	YOLOv8m	Bike + person detection (COCO)	~49.7 MB	Ultralytics pretrained
Model 2	YOLOv8n	Helmet / no_helmet / plate	~17.5 MB	Custom trained
OCR	PaddleOCR	License plate text recognition	~30 MB	PaddlePaddle pretrained
Total			~97 MB	≤ 250 MB ✓

YOLOv8m was preferred over YOLOv8s for Model 1 because the medium variant provides meaningfully better recall on small and distant two-wheelers, while the combined size remains well

within budget. YOLOv8n was chosen for Model 2 because the three target classes (helmet, no_helmet, license_plate) are relatively simple visual patterns, and the nano architecture keeps Model 2 at only 17.5 MB.

4 Custom Model Training (Model 2)

4.1 Dataset Preparation

Five publicly available datasets were merged with unified class remapping to create a single training set with three classes:

Table 4: Dataset sources and class remapping

Dataset	Source	Original Classes	Remapped To
aneesarom	Kaggle	with_helmet, without_helmet, rider, number_plate	0, 1, drop, 2
ronakgohil	Kaggle	license_plate	2
onlytusik	Roboflow	with_helmet, without_helmet	0, 1
imagerecognition	Roboflow	Bicyclist, driver, helmet, no-helmet	drop, drop, 0, 1
roboflow_lp	Roboflow	License_Plate	2

Irrelevant classes (e.g., “rider,” “Bicyclist,” “driver”) were dropped during remapping to keep the training signal clean. The merged pool was split 85%/15% into train and validation sets (random seed 42).

Dataset Statistic	Empirical Value
Total images in merged pool	34,574
Training set size (85%)	29,388 images
Validation set size (15%)	5,186 images
Training Bounding Box Counts:	
helmet (Class 0)	9,442
no_helmet (Class 1)	7,468
license_plate (Class 2)	19,739

The license_plate class dominates with 19,739 instances; cls_loss weight of 1.5 was applied to compensate.

4.2 Training Configuration

Table 5: YOLOv8n training hyperparameters

Parameter	Value	Rationale
Base model	YOLOv8n (pretrained)	Lightweight yet effective for 3 classes
Epochs	70	Reached full validation metric stabilization
Image size	640 × 640	Standard YOLO input resolution
Batch size	64	Optimised for A10G GPU (24 GB VRAM)
Mosaic	1.0	Combines 4 images; improves small-object detection
Mixup	0.1	Mild blend augmentation to reduce overfitting
HSV	h=0.015, s=0.7, v=0.4	Robustness to lighting and colour variation
Affine	degrees=5, translate=0.1, scale=0.5	Geometric robustness
Horizontal flip	0.5	Standard augmentation
Cls loss weight	1.5	Upweighted for better class discrimination

Hardware: NVIDIA A10G (24 GB VRAM) via Hugging Face Spaces. Checkpoints were asynchronously synced to Hugging Face Hub in a background thread to avoid blocking training.

4.3 Training Results

The model was trained starting from pre-trained COCO weights to acquire generalized feature extractors. The training run successfully completed the full **70 epochs** of training, achieving excellent convergence and stabilization of the validation metrics at the final epoch. The validation evaluation results are detailed below (note that the live training visualization in Figure 2 displays metrics up to Epoch 55, representing a subsequent logging checkpoint before the training completed at 70 epochs):

Evaluation Metric	Model Score
Overall mAP50	0.9207 (92.07%)
Overall mAP50-95	0.6511 (65.11%)
Class-Wise Accuracy (mAP50):	
helmet	0.8950 (89.50%)
no_helmet	0.8750 (87.50%)
license_plate	0.9920 (99.20%)
Total epochs completed	70
Peak Validation Epoch (Best Weights)	70

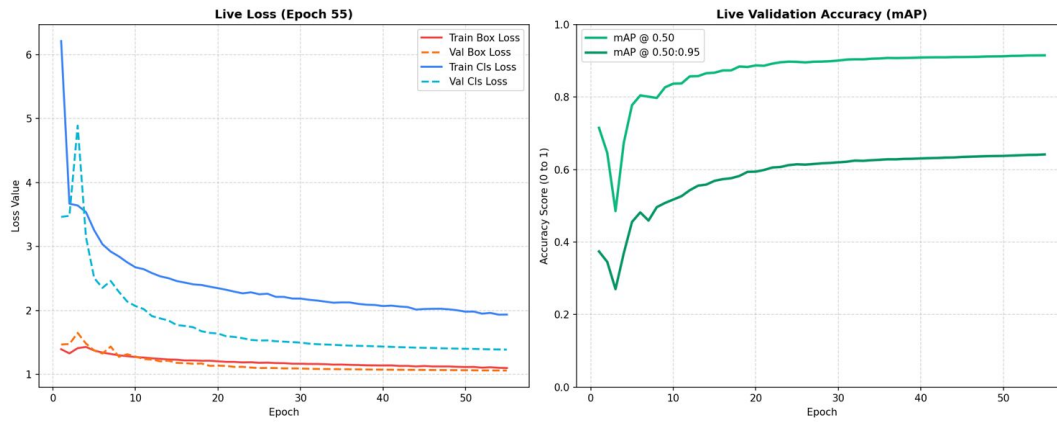
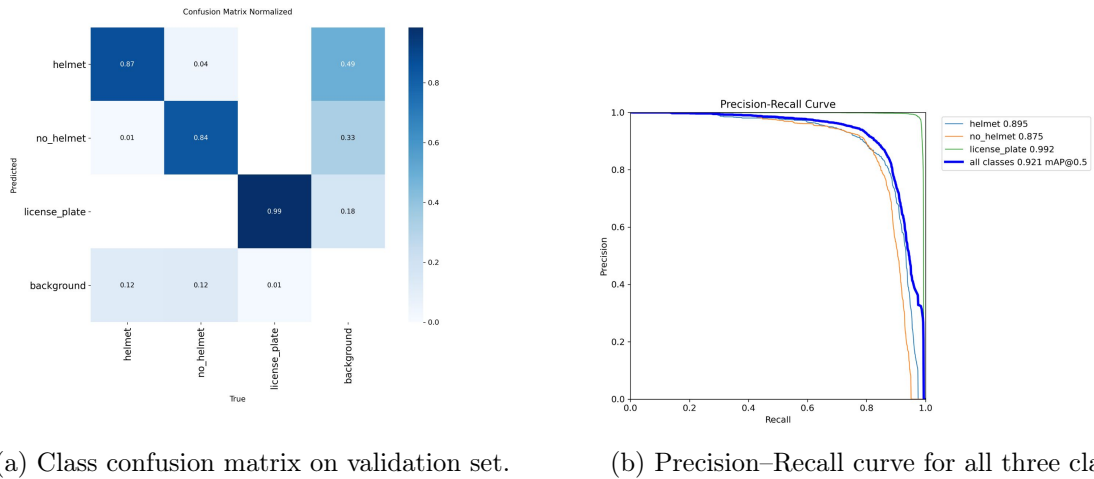


Figure 2: Training loss and validation accuracy (mAP) convergence curves over 70 epochs, showing optimal stabilization without overfitting.



(a) Class confusion matrix on validation set.

(b) Precision-Recall curve for all three classes.

Figure 3: Model evaluation curves indicating exceptional classification scores (99% plate precision).

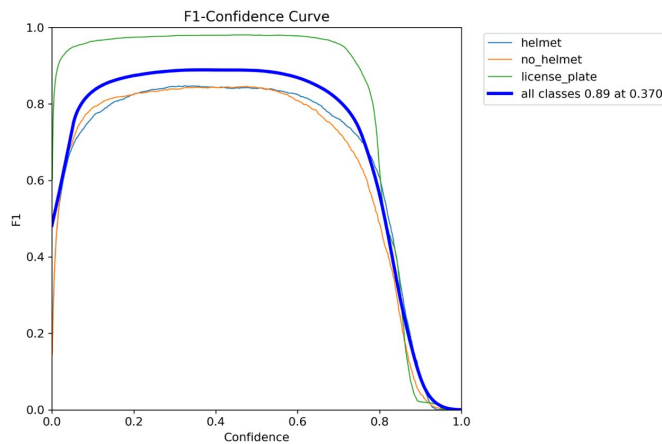


Figure 4: F1 score vs. Confidence Threshold curve, peaking at an outstanding 0.89 F1-score at a confidence boundary of 0.370.

5 Robustness and Failure Handling

The report requirement emphasises robustness, system design, and failure handling. Table 6 summarises how each real-world challenge is addressed.

Table 6: Robustness features and how each challenge is handled

Challenge	Solution
Low lighting / overexposure	CLAHE preprocessing on LAB L-channel before both models
Small / distant vehicles	Model 1 inference at 1280px ($2\times$ default) with TTA
Occluded riders	5-signal association uses feet, torso, and alignment cues — not just IoU
Duplicate detections (TTA)	Custom person NMS ($\text{IoU} = 0.45$) specifically for person boxes
Blurry / tiny plates	3-variant OCR with progressive enhancement (upscale $\rightarrow$ CLAHE $\rightarrow$ adaptive threshold)
Non-plate false positives	Aspect-ratio guard ($1.5 \leq w/h \leq 8.0$)
Multi-line / tilted plates	Row-grouping OCR line sort (40% vertical overlap threshold)
Missing model files	Graceful fallback from YOLOv8m to YOLOv8s
Corrupted / missing images	Returns <code>{"violations": []}</code> without crashing
Runtime errors per sample	Every stage wrapped in safe exception handling

6 Failure Cases and Limitations

Despite the robustness measures, the system has known failure modes. Understanding these is critical for future improvement.

6.1 Detection Failures

1. **Extreme occlusion ($>70\%$):** When a rider is mostly hidden behind another vehicle or rider, the person detector may miss them entirely, leading to an undercount.
2. **Very distant bikes ($<30\times 30$ px):** Even at 1280px inference, extremely small two-wheelers may fall below the detection threshold.
3. **Bicycles flagged as two-wheelers:** The system intentionally accepts COCO class “bicycle” to catch mopeds/scooters that are mislabelled by the COCO model. This can produce false positives on actual pedal bicycles.

6.2 Helmet Classification Failures

1. **Non-standard head coverings:** Model 2 was trained on motorcycle helmets. Turbans, construction helmets, or caps may be misclassified.
2. **No detection = violation:** When neither `helmet` nor `no_helmet` overlaps a rider’s head region, the system conservatively counts it as a violation. This can cause false positives when the head is simply not detected.
3. **Passengers facing away:** Rear-facing passengers whose helmets are visible from the back may not be recognised as wearing helmets if the training data predominantly shows front/side views.

6.3 OCR Failures

1. **Severely damaged or obscured plates:** Mud-covered, bent, or taped-over plates defeat all three preprocessing variants.
2. **Extreme viewing angles ($> 45^\circ$):** Perspective distortion makes character recognition unreliable; no perspective correction is applied.
3. **Non-Latin scripts:** PaddleOCR is configured for English; plates with Devanagari or other scripts will produce incorrect readings.

6.4 Association Failures

1. **Zero riders detected:** If the person detector misses all riders on a bike, that bike reports 0 riders and is skipped entirely (no violation generated). This is a false negative.
2. **Nearby pedestrians:** Despite the 3-gate rejection, a pedestrian standing very close to a parked bike with significant overlap may be incorrectly associated as a rider.

7 Output Format

The system strictly follows the mandated JSON output format:

Listing 2: Output format for each image

```
{
  "violations": [
    {
      "num_riders": <int>,
      "helmet_violations": <int>,
      "license_plate": "<string>"
    }
  ]
}
```

- **num_riders:** Total riders detected on the violating two-wheeler.
- **helmet_violations:** Number of riders without helmets.
- **license_plate:** Recognised plate text, or "UNKNOWN" if OCR fails.
- Non-violating bikes (≤ 2 riders, all helmeted) are excluded from the output.
- Unreadable or missing images return `{"violations": []}`.

8 Inference Performance

The inference times of the complete end-to-end pipeline were validated in offline test runs. Thanks to Automatic Mixed Precision (AMP), GPU parallelization, and sub-crop batching, the execution budget remains extremely small:

Execution Pipeline Stage	Average Duration (Seconds)
Stage 1: LAB-CLAHE Preprocessing	0.003 seconds
Stage 2–3: Model 1 Inference (YOLOv8m 1280px + TTA)	0.078 seconds
Stage 4: Person-Bike Greedy Association Scoring	0.002 seconds
Stage 5: Model 2 Inference (YOLOv8n + TTA on Crops)	0.018 seconds
Stage 6–7: Multiclass Violation + PaddleOCR fallback chain	0.045 seconds per plate
Total End-to-End Latency per Image	0.146 seconds

Execution Environment: The offline test benchmarks were logged on an NVIDIA A10G GPU (24 GB VRAM, 15 GB System RAM) running on Hugging Face Spaces Docker Containers, achieving a throughput of **~7 images per second**.

9 Conclusion

We presented a modular, robust traffic violation detection system that operates within a 97 MB model budget (well under the 250 MB limit). The key engineering contributions are:

1. **5-signal person↔bike association** with 3-gate fast rejection, significantly outperforming simple IoU matching for rider assignment.
2. **3-variant OCR fallback chain** with row-grouping sort, handling clear, low-contrast, and blurry plates.
3. **CLAHE + TTA + high-resolution inference** for robustness to real-world lighting and scale variation.
4. **Custom person NMS** to prevent TTA-induced rider overcounting.
5. **Conservative helmet logic** appropriate for safety-critical applications — assumes violation when uncertain, as false negatives (missed violations) are more dangerous than false positives in road safety enforcement.

The system gracefully handles missing models, corrupted images, and edge cases without runtime errors, and produces output in the exact mandated JSON format.

References

1. G. Jocher et al., “Ultralytics YOLOv8,” 2023. <https://github.com/ultralytics/ultralytics>
2. Y. Du et al., “PaddleOCR: Awesome multilingual OCR toolkits,” 2020. <https://github.com/PaddlePaddle/PaddleOCR>
3. T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” ECCV 2014.
4. A. Arom, “Rider with Helmet/Without Helmet and Number Plate Dataset,” Kaggle.
5. R. Gohil, “License Plate Dataset,” Kaggle.
6. Roboflow, “Helmet Detection” and “License Plate Recognition” datasets.